Roteiro de Apresentação — Obra Fácil (Entrega G2)

Público: Professor da disciplina de Engenharia de Software — PUC-MG, 2026/1.
Duração sugerida: 15–20 minutos.
Formato: demo ao vivo do produto + walkthrough do código quando fizer sentido.

Este roteiro guia a apresentação endereçando, item por item, a rubrica da Entrega G1 (nota inicial **45/100**) e mostrando como cada ponto foi endereçado na G2. Mantém o tom direto: produto funcionando primeiro, explicação por cima.

0. Setup antes da apresentação

Abas do navegador prontas:

1. https://app-devai-frontend.vercel.app (frontend)
2. https://app-devai-backend.vercel.app/api/docs (Swagger)
3. https://app-devai-backend.vercel.app/api/health (saúde)
4. GitHub do projeto: https://github.com/lexcesar/obra-facil
5. Aba com `docs/gap-analysis-entrega-g1.md` aberto (mostra a análise honesta dos gaps da G1)
6. Aba com `docs/entrega-g2-implementacao.md` aberto (mapeia rubrica → implementação)

Login nas 2 contas preparadas:

- Sessão 1 do browser: cliente
- Sessão 2 (janela anônima): profissional

Faça um smoke test 15 minutos antes. Garante que backend + frontend estão verdes, credenciais funcionam, e `ANTHROPIC_API_KEY` está ativa em produção (teste a cotação com IA).

1. Abertura (1 min)

> "Na G1 tiramos 45/100. A banca apontou bug em 'Meus Pedidos', ausência de fluxo profissional, ausência de IA real, faltavam observabilidade, testes e IaC. Vamos mostrar como cada ponto foi endereçado."

Mostrar rapidamente o `docs/gap-analysis-entrega-g1.md` para evidenciar que o time **leu a rubrica, classificou cada item e priorizou por esforço × impacto**.

2. RF1 — Meus Pedidos (bug crítico) (2 min)

Observação original da banca: *"Histórico traz dados de outros usuários."*

Demo

1. Fazer login como **cliente A**
2. Entrar em `/pedidos` — mostrar que lista pedidos (se houver no banco de prod)
3. Abrir `docs/plans/2026-04-18-001-fix-meus-pedidos-isolamento-usuario-plan.md` brevemente para mostrar a análise do root cause

O que dizer

> "O bug estava no `ClerkAuthGuard` em modo bypass: retornava o primeiro perfil do banco sem `ORDER BY`, colapsando todos os usuários no mesmo `profile.id`. Corrigimos com resolução determinística via header `X-Dev-User-Id` e `ORDER BY` no fallback, além de adicionar JIT provisioning idempotente com o webhook Clerk."

Evidências

- Commit: `apps/backend/src/core/guards/clerk-auth.guard.ts`
- Testes: `apps/backend/test/orders.e2e-spec.ts` — cenários de isolamento bidirecional contra Postgres real
- CI roda esses testes a cada push

3. RF2 — Fluxo profissional (3 min)

Observação original: *"Pendente disponibilizar credenciais com perfil do profissional."*

Demo

1. Trocar para a sessão do **profissional**
2. Entrar em `/profissional/dashboard`
3. Mostrar os 4 cards de stats (visitas, obras ativas, conversas, concluídas)
4. Se houver obra agendada: clicar **Iniciar obra** → mostrar transição
5. Se houver obra ativa: clicar **Marcar como concluída** → mostrar transição + `progress_pct=100`
6. Se houver visita confirmada: clicar **Concluir** ou **Cancelar**

O que dizer

> "O profissional tem um dashboard com stats e ações ponta-a-ponta. Cada ação é protegida por um guard que confere `role='professional'` **e** que o recurso pertence ao profissional logado — dois profissionais não conseguem mexer na obra um do outro."

Evidências no Swagger

Abrir https://app-devai-backend.vercel.app/api/docs:

- `GET /v1/professionals/me/dashboard`
- `PATCH /v1/works/:id/start`, `PATCH /v1/works/:id/complete`, `PATCH /v1/works/:id/progress`
- `PATCH /v1/visits/:id/cancel`, `PATCH /v1/visits/:id/complete`

4. RF3 — Tecnologia de fronteira (IA) (3 min)

Observação original: *"Nenhuma tecnologia avançada implementada."*

Demo

1. Ainda como cliente, acessar `/cotacao/ia`
2. Digitar: **"Reformar banheiro pequeno de 4m² com troca de piso, azulejo, louças e metais"**
3. Clicar **Gerar lista de materiais**
4. Aguardar ~5 segundos

5. Mostrar a lista gerada (20+ itens realistas com quantidades, unidades e categorias)

O que dizer

> "Integração **real** com Anthropic Claude Haiku 4.5 — não é mock. O prompt é especialista em materiais de construção civil no Brasil. Resposta validada por schema JSON, itens categorizados, observações contextuais. Tudo logado com tokens usados via Pino."

Evidências

- `apps/backend/src/modules/ai/ai.service.ts`
- Teste unitário com mock do SDK: `ai.service.spec.ts`
- Swagger: `POST /v1/ai/material-quote`
- No health check: `env: production`

5. RNF-04 — Observabilidade (1 min)

Demo

1. Abrir https://app-devai-backend.vercel.app/api/health
2. Mostrar a resposta JSON:

```
{
  "data": {
    "status": "ok",
    "db": "ok",
    "uptime_s": 264,
    "response_ms": 617,
    "version": "dev",
    "env": "production",
    "timestamp": "..."
  }
}
```

O que dizer

> "Endpoint `/api/health` testa a conexão real com o banco. Logs estruturados em JSON via `nestjs-pino`, com request ID automático para correlação e redact de cabeçalhos sensíveis

> (Authorization, Cookie, X-Dev-User-Id)."

Evidência no código

- `apps/backend/src/app.module.ts` — configuração do Pino com redact
- `apps/backend/src/modules/health/health.controller.ts`

6. RNF-05 — Testabilidade (2 min)

De 1 teste placeholder (Hello World) para 68 testes.

Demo

No terminal (pré-aquecido):

```
npm test --workspace=backend          # 49 testes Jest em ~0.5s
npm test --workspace=frontend         # 9 testes Vitest em ~0.8s
```

Ou abrir o CI: https://github.com/lexcesar/obra-facil/actions — mostrar o último run verde.

O que dizer

> "Quatro frameworks, cada um no seu lugar:
>
> - **Jest** para unit e integração do backend (49 + 6 e2e).
> - **Vitest + Testing Library** para unit do frontend (9).
> - **Playwright** para e2e browser (4).
> - Tudo roda no CI a cada push, incluindo container Postgres com schema e seed aplicados."

Evidência

- `.github/workflows/ci.yml`
- `docs/entrega-g2-implementacao.md` seção RNF-05 com breakdown por arquivo

7. RNF-06 — IaC (1 min)

Observação original: *"Artefatos IaC não identificados."*

O que dizer

> "Três camadas de IaC:
>
> 1. **Containers OCI** (Dockerfile + docker-compose) para runtime local/self-hosted
> 2. **Schema SQL versionado** (`docker/01-schema.sql` , `docker/02-seed.sql`) — banco reproduzível
> 3. **Terraform** em `infra/terraform/vercel/` declarando os dois projetos Vercel (backend + frontend) com env vars e integração GitHub"

Evidência

- Mostrar `infra/terraform/vercel/main.tf`
- Mostrar `docs/terraform-plan-output.txt` — saída real do `terraform plan` comprovando que a declaração é válida

8. Segurança reforçada (1 min, bônus)

Na mesma rodada de correções:

- **Startup guard** (`apps/backend/src/main.ts`): backend aborta com exit 1 se `NODE_ENV=production` e `DISABLE_CLERK_AUTH=true` simultaneamente
- **Error messages sanitizadas**: 401 devolve `"Não autorizado"` sem vazar `clerk_id` tentado (evita enumeração)
- **Header X-Dev-User-Id**: ignorado em produção por verificação dupla de env var

> "Esses são bônus que vieram de um code review adversarial automatizado — o time roda reviews estruturados antes de cada PR."

9. Processo de engenharia (1 min)

Mostrar rapidamente o GitHub do projeto:

- PRs #36, #37, #38 (linha do tempo de como a G2 foi entregue)
- Histórico de commits com Conventional Commits
- CI badge verde
- Plano detalhado versionado em `docs/plans/`

> "A entrega foi estruturada como três PRs incrementais: o primeiro fecha o bug crítico + 5 dos 6 itens da rubrica, o segundo completa o fluxo profissional com ações, o terceiro adiciona o artefato de Terraform plan. Cada PR passa pela mesma pipeline."

10. Fechamento (1 min)

Tabela resumo (`docs/entrega-g2-implementacao.md` tem isso ao final):

Antes (G1)	Depois (G2)
45/100 pontos	Potencial ~100/100
1 teste	68 testes em 4 frameworks
Logs `console.log`	Pino JSON + requestId + redact
Sem IaC	Terraform + Docker + SQL
Sem IA	Claude Haiku 4.5 integrado
Bug crítico em Meus Pedidos	Corrigido + testes de isolamento
Sem dashboard profissional	Dashboard + ações ponta-a-ponta

> "Recap rápido: endereçamos os seis gaps apontados pela banca; as implementações estão em produção na Vercel e o código tem 68 testes automatizados. Qualquer pergunta?"

Perguntas prováveis da banca + respostas prontas

P: "O webhook do Clerk está funcionando em produção?"
R: Sim. `CLERK_WEBHOOK_SECRET` está configurado no Vercel. Temos JIT provisioning como fallback no `ClerkAuthGuard` caso o webhook falhe ou demore.

P: "Como vocês garantem que o usuário A não vê pedidos do B?"
R: Três camadas: (1) SQL com `WHERE client_id = $1` ; (2) `OrdersController` exige `role='client'` e passa `profile.id` do guard; (3) teste e2e em `orders.e2e-spec.ts` prova isolamento bidirecional contra Postgres real.

P: "A IA pode inventar coisas (alucinar)?"
R: Sim, é um risco inerente. Mitigações: prompt sistêmico estrito em português, schema JSON validado no backend, descrição limitada a 2000 caracteres. Para MVP é aceitável; próximo passo seria adicionar price grounding contra as lojas reais.

P: "Por que Terraform só para Vercel e não Supabase?"
R: O provider oficial do Supabase gerencia apenas projeto/branches — não cria tabelas, RLS ou schemas. Como o schema já está versionado em `docker/01-schema.sql` , o ganho marginal não compensa a complexidade adicional. Decisão documentada no plan.

P: "Como escalariam para mil usuários?"
R: Vercel serverless escala horizontal automático. Supabase pooler suporta até ~200 conexões simultâneas no free tier. Cache aggressive para leituras de profissionais (30s+). Próximos gargalos seriam: IA latency (mover pra fila/SSE) e cold starts.

P: "Onde está o monitoramento de erros (Sentry, DataDog)?"
R: Não temos APM externo — usamos Pino com JSON logs que a Vercel agrega. Para próximas iterações, `nestjs-pino` já exporta para qualquer OTLP collector.

P: "Vocês usaram alguma LLM para gerar o código?"
R: Sim — Claude Code como par de programação. Todos os PRs passaram por revisão humana, CI, e a arquitetura foi decidida pelo time. A IA acelerou implementação, não substituiu decisões.

Se algo falhar durante a demo

- **Backend 500**: verificar `https://app-devai-backend.vercel.app/api/health` — se DB estiver down, plano B é fazer o passeio pelo Swagger + código
- **IA demora ou trava**: mostrar log de chamada no Vercel + cache de response de uma execução anterior (screenshot)
- **Login Clerk trava**: usar a aba já logada que você preparou no setup
- **Vercel rollback necessário**: `git revert` + push; deploy novo em ~1min

Artefatos para entregar junto

1. **Link do repositório**: https://github.com/lexcesar/obra-facil
2. **URL da aplicação**: https://app-devai-frontend.vercel.app
3. **Credenciais** (cliente + profissional — entregar por canal seguro, não neste doc)
4. **Esta pasta de docs**: `docs/` contém gap analysis, plano técnico, mapeamento rubrica→implementação, terraform plan, roteiro desta apresentação

Última atualização: 2026-04-18